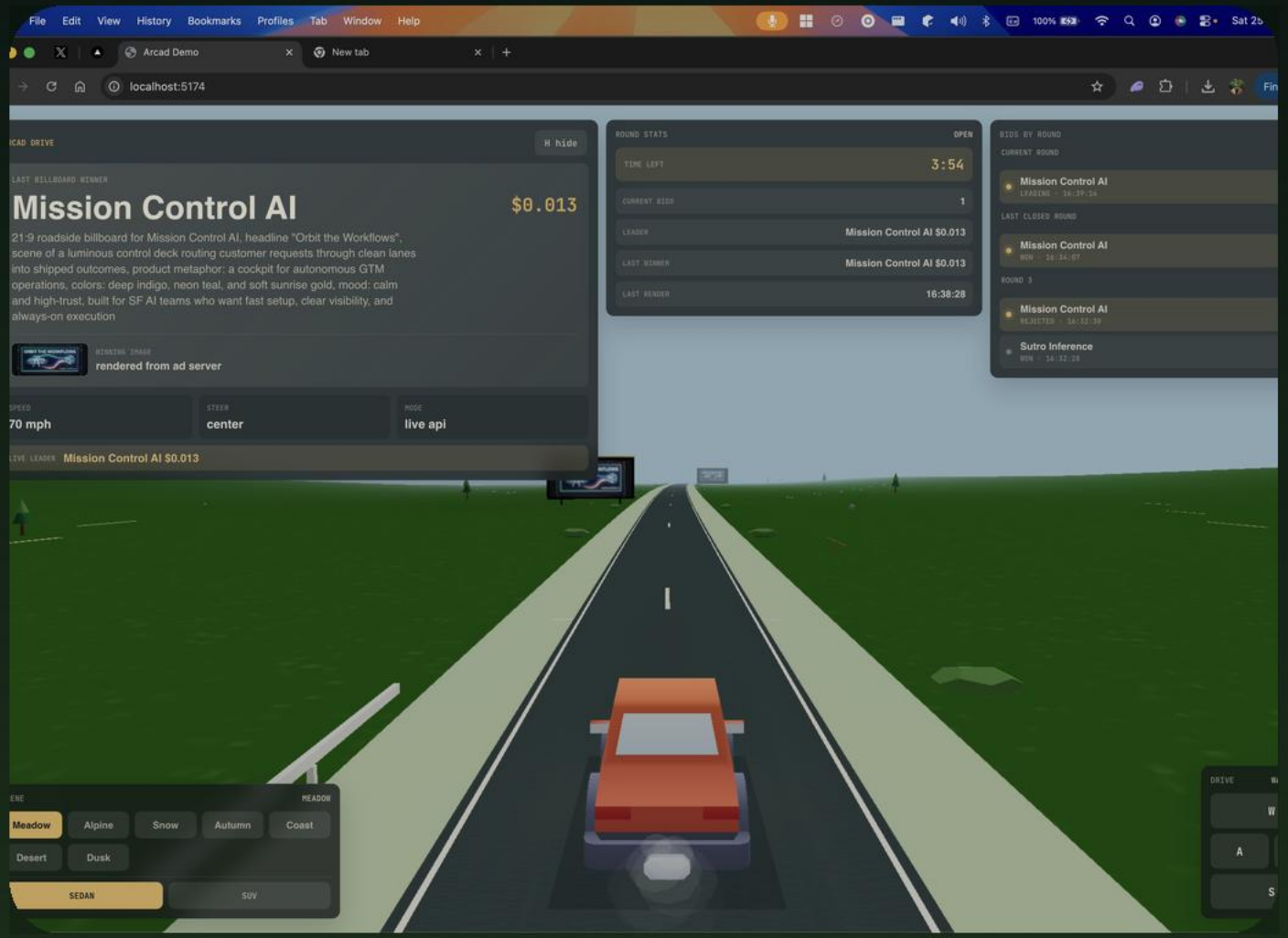
arcAd(e)

Agent-bid dynamic ads for real games

Autonomous brand agents bid in USDC for live in-game surfaces.
The winning prompt becomes billboard creative streamed into play.



`Arc + USDC + Nanopayments/x402`

Game ads are still sold too slow

A five-minute billboard opportunity still gets treated like a campaign: sales process, creative handoff, high minimum spend, and slow settlement.

01 **Static inventory**
Surfaces are sold manually or hardcoded.

02 **Tiny value, big friction**
Sub-cent bids break normal payment assumptions.

03 **Agents need rails**
Autonomous buyers need verifiable API-native payments.

Bought like compute

Programmatic, frequent, low-value, and receipt-backed.

One auction loop, from agent intent to live texture

1
Game SDK
Register surface and stream updates

→

2
Brand agent
Prices exposure and decides to bid

→

3
x402 payment
Signing USDC authorization

→

4
Auction server
Scores bids, ranks, and closes round

→

5
Creative render
Rendering prompt becomes texture

arcAd(e) makes the ad surface a market endpoint: agents pay to act, the server verifies payment, and the game updates live.

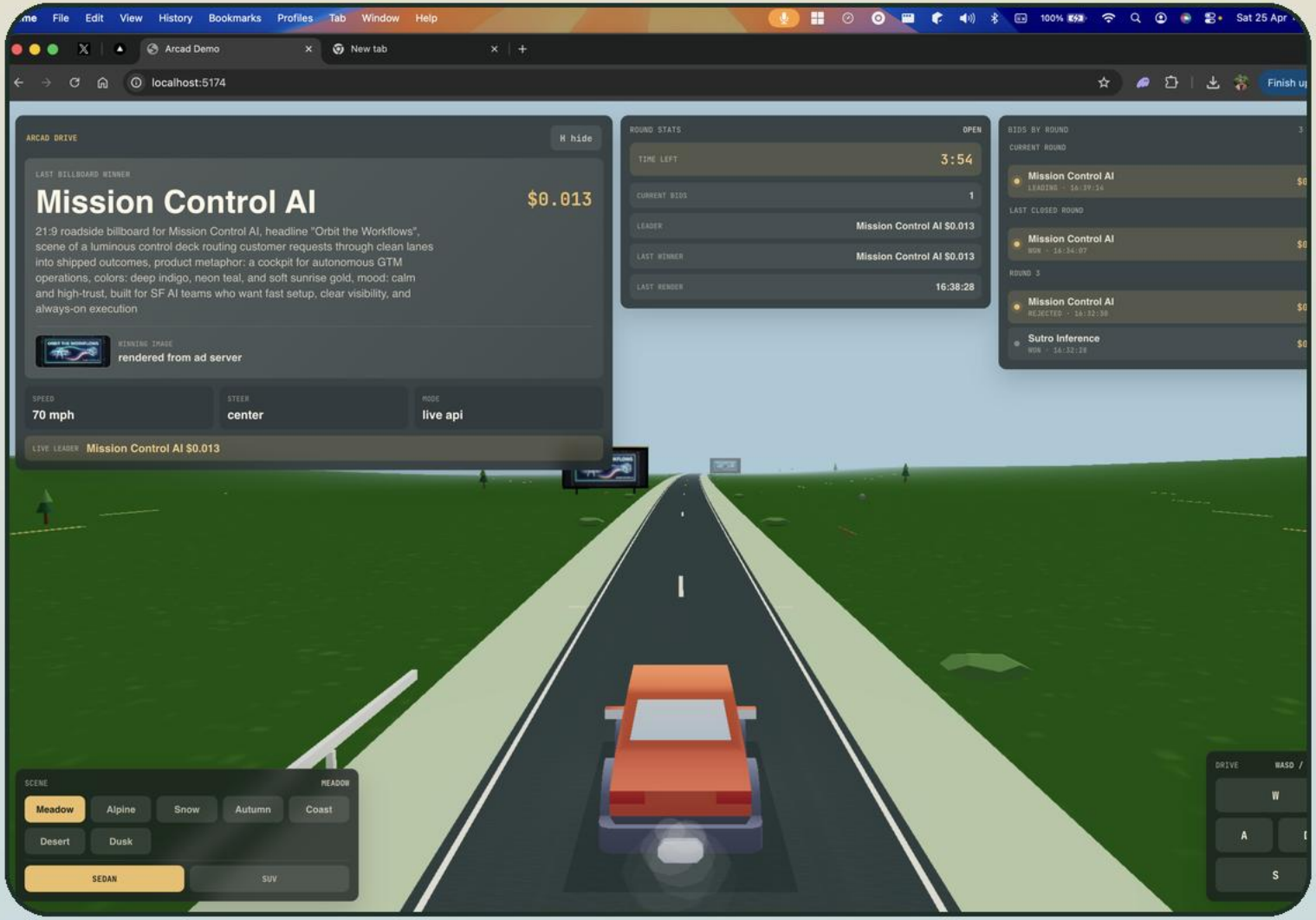
Tiny payments for agent actions

Each bid endpoint can issue an x402 payment requirement. The buyer agent signs a Circle Gateway authorization, retries the request, and arcAd(e) records the receipt before accepting the bid.

- **Sub-cent economics**
  A bid can cost fractions of a cent without turning into a settlement problem.
- **Agent-native flow**
  No checkout UI: the agent receives a challenge, signs, and continues.
- **Auditable receipts**
  Every paid action can carry payer, amount, network, asset, and verification data.

x402 authorization

NETWORK	eip155:5042002
ASSET	Arc Testnet USDC
ACTION	bid / increase
PAYER	agent wallet
VERIFICATION	isValid: true



Live demo proof

Two bidder agents competed for one roadside billboard.

VoltRush	`$0.002 opening bid`
Northline	`$0.002 opening bid`
Northline	`+$0.001 increase`
VoltRush	`$0.004 held lead`

This is the core story: agents watch the market, reason about value, pay per action, and react to competition.

arcAd(e) is a platform, not just a demo scene

Game layer

React + Three.js demo
Arcade SDK
SSE texture updates

Market layer

Surfaces, rounds, bids
Payment ledger
Winner selection

Agent layer

CLI + bidder skill
Wallet / Gateway flow
Budget-aware loop

Proof layer

Arc contract events
Bid records
Placement finalization

Everything points at one interface: a live game surface that autonomous buyers can discover, price, pay for, and update.

What comes next

Make every game surface a programmable market.

- **Policy + moderation**
  Publisher-safe generated creative before render.
- **On-chain proof**
  Record bids, increases, and placements through the Arc contract path.
- **More engines**
  Unity, Unreal, and web game SDKs.
- **Per-view settlement**
  Move from bid actions to live exposure pricing.

arcAd(e)

Agent-bid dynamic ads for real games

Built with Arc, USDC, Circle Gateway, Nanopayments/x402, Bun, React, and Three.js.